

UTEC

UNIVERSIDAD DE INGENIERÍA
Y TECNOLOGÍA

MINIA MAUTA

Informe de proyecto (parte 1)

Robot educativo inteligente para la comprensión lectora

Proyectos Interdisciplinarios 2

Profesor: Fabrizio Julio Castillo Castillo

Periodo: 2026-2

Alumnos:

- Guillermo Guerrero Sotomayor
- Robel Francisco Lévano Mallqui
- Rocio Alessandra Cutire Gomez
- Treicy Calsina Gutierrez
- Dominguez Valentin Miguel Angel

1. Problemática y solución

1. ¿Qué problema específico van a resolver y para quién?

En diversas zonas rurales y alejadas de los centros urbanos del Perú, los niños pueden enfrentar dificultades para desarrollar adecuadamente sus habilidades de comprensión lectora debido a factores como el acceso limitado a recursos educativos, materiales de apoyo y acompañamiento personalizado durante el proceso de aprendizaje.

Estas dificultades pueden afectar su capacidad para identificar información explícita en un texto, reconocer ideas principales, realizar inferencias y comprender el significado general de una lectura. Si estas habilidades no se fortalecen durante la educación primaria, pueden repercutir posteriormente en el aprendizaje de otras áreas académicas.

Público objetivo: MinIA Mauta está dirigido principalmente a niños de educación primaria que ya han adquirido la capacidad básica de leer, pero que requieren reforzar sus habilidades de comprensión lectora. El sistema busca proporcionar una herramienta interactiva de apoyo que permita al niño responder preguntas sobre textos previamente leídos y recibir retroalimentación inmediata sobre sus respuestas.

2. ¿Cuál es su solución en una frase?

Solución (MVP): MinIA Mauta es un dispositivo educativo interactivo que refuerza la comprensión lectora mediante preguntas orales sobre textos predeterminados, evaluación de las respuestas del niño y retroalimentación inmediata mediante voz.

- **Interacción por voz:** después de que el niño lee un texto previamente establecido, MinIA Mauta formula oralmente preguntas de comprensión relacionadas con la lectura y escucha las respuestas del niño mediante un micrófono.
- **Evaluación de comprensión:** el sistema analiza la respuesta del niño utilizando inteligencia artificial para determinar si es correcta o incorrecta.
- **Retroalimentación inmediata:** si la respuesta es correcta, MinIA Mauta proporciona un mensaje de refuerzo positivo y continúa con la siguiente pregunta. Si la respuesta es incorrecta, orienta al niño indicando la línea o sección del texto donde puede encontrar la información necesaria y le brinda una segunda oportunidad para responder.
- **Apoyo ante un segundo error:** si el niño vuelve a responder incorrectamente, MinIA Mauta proporciona la respuesta correcta y continúa con la siguiente pregunta.
- **Actividad estructurada:** cada texto cuenta con un banco de diez preguntas de comprensión, de las cuales el sistema selecciona cinco para cada sesión. Al finalizar las cinco preguntas, MinIA Mauta comunica al niño que la actividad ha terminado y finaliza la sesión.

Solución final: en una versión posterior, MinIA Mauta podrá incorporar una cámara para reconocer diferentes textos físicos y determinar automáticamente su contenido y estructura. Mediante inteligencia artificial integrada y una base de datos local, el dispositivo podrá identificar la información relevante dentro de la lectura, formular o seleccionar preguntas de comprensión y

orientar al niño hacia la ubicación de las respuestas, reduciendo progresivamente la dependencia de servicios externos y de una conexión a Internet.

- **Aprendizaje adaptativo:** futuras versiones podrán ajustar el nivel de dificultad de las preguntas de comprensión según el desempeño del niño durante las diferentes sesiones.

2. Objetivos

Objetivo general:
Diseñar e implementar un robot educativo inteligente que, mediante interacción por voz e inteligencia artificial, refuerce la comprensión lectora en niños de educación primaria a través de preguntas sobre textos predeterminados, evaluación de sus respuestas y retroalimentación inmediata.

Objetivos específicos:

- Diseñar e implementar el sistema de interacción por voz del robot, incluyendo la captura de las respuestas del niño mediante micrófono y la reproducción de preguntas y retroalimentación mediante un altavoz.
- Implementar un sistema de comunicación entre el ESP32 y un servidor encargado de procesar el audio, utilizar servicios de inteligencia artificial para interpretar y evaluar las respuestas del niño y generar la retroalimentación correspondiente.
- Implementar la lógica de evaluación de comprensión lectora, de manera que el sistema determine si la respuesta del niño es correcta o incorrecta y actúe según el resultado.
- Implementar un mecanismo de apoyo progresivo: ante una primera respuesta incorrecta, el sistema indicará la línea o sección del texto donde se encuentra la información necesaria; si el niño vuelve a responder incorrectamente, proporcionará la respuesta correcta antes de continuar con la siguiente pregunta.
- Implementar una sesión de evaluación basada en textos predeterminados, cada uno con un banco de diez preguntas, de las cuales el sistema seleccionará cinco para ser respondidas durante cada actividad.
- Asegurar que, en una versión final, el robot pueda incorporar una cámara para reconocer textos físicos, identificar la ubicación de la información relevante y operar con mayor autonomía, reduciendo progresivamente la dependencia de servicios externos y de conexión a Internet.

3. Conceptualización

GARAGE:

El área GARAGE se encarga de la plataforma de hardware y del firmware del robot: define los sensores, actuadores y microprocesadores, implementa el cableado y valida el funcionamiento físico del circuito de extremo a extremo.

MVP: microprocesador CHIP ESP32-NodeMCU con micrófono (sensor) y altavoces (actuador). Define el uso de audio PCM16 a 16 kHz por streaming I2S+DMA y el modo half-duplex (escucha/parlante) para evitar el eco.

Final: microprocesador CHIP Raspberry Pi 5 8GB con micrófono y cámara (sensores) y altavoces y pantalla (actuadores); el mayor cómputo habilita la IA local y el aprendizaje adaptativo.

- **Entregables:** firmware con las funciones de integración (escuchar_micro(), llamarAPI(), escucharAPI(), reproducir_parlante()), montaje del circuito (sección 4.1) y pruebas de audio **voz → servidor → voz** bajo el contrato definido con CS.

FABLAB:

El área FABLAB se encarga del diseño físico, la carrocería y la presentación del robot.

- **Forma de búho:** detalles de un búho para llamar la atención de los niños y dar al dispositivo una identidad amigable.
- **Usabilidad y seguridad:** diseño amigable, sin bordes ásperos y con materiales no tóxicos para fomentar su uso por menores.
- **Control simple:** solamente un botón de encendido y apagado (switch SPST, sección 4.1).
- **Energía autónoma:** batería recargable y panel solar para operar en aulas remotas sin red eléctrica.

CS:

El área Computer Science se encarga del servidor, la lógica de IA y los datos del sistema.

- **Servidor en Spring Boot:** actúa como puente (proxy) entre el ESP32 y la IA; aloja la clave de la API y abstrae el modelo de IA detrás de una interfaz única.
- **Computadora que aloja el servidor:** ejecuta Spring Boot en el aula y concentra el ASR, la transcodificación y el TTS.
- **URL de la API integrada en el ESP32:** el firmware llama al endpoint del servidor con el audio capturado (llamarAPI(audio)).
- **APIs de agentes de IA (Gemini Live API de Google):** generan preguntas, evalúan las respuestas y brindan retroalimentación por voz.
- **Base de datos local (versión final):** registra el progreso de cada niño y soporta el aprendizaje adaptativo

Requisitos de integración (CS → GARAGE): el equipo de Computer Science exige que el firmware del ESP32 exponga las funciones de comunicación con el hardware y con el servidor, de modo que la interacción por voz quede totalmente controlada por código:

- **escuchar_micro():** captura el audio del usuario a través del micrófono.

- **llamarAPI(audio):** envía el audio capturado al servidor Spring Boot mediante la URL de la API integrada en el ESP32.
- **escucharAPI():** espera y recibe la respuesta (audio) generada por el servidor.
- **reproducir_parlante():** reproduce por el altavoz la respuesta recibida del servidor.

Arquitectura de comunicación: el flujo del MVP es ESP32 → servidor Spring Boot → API de IA (Gemini Live API) → servidor (conversión de respuesta a audio, TTS) → ESP32. La consulta a la IA no se realiza dentro del ESP32 porque el microprocesador cuenta con recursos limitados de memoria y procesamiento para el streaming de audio y el manejo SSL/TLS, además de que centralizar la llamada en el servidor mantiene la clave de la API protegida en la computadora que lo aloja.

4. Prototipo (componentes) y funcionalidad

4.1. Lista de componentes (Hardware y montaje)

Componentes definidos para el prototipo MVP con su cantidad y función dentro del sistema:

Componente	Cant.	Función en el sistema
ESP32 NodeMCU DevKit v1 (30 pines)	1	Microcontrolador principal: gestiona Wi-Fi, comunicación I2S de audio y la llamada a la API del servidor.
Módulo micrófono I2S INMP441	1	Capta la voz del niño con salida digital; alimenta la función escuchar_micro().
Módulo amplificador I2S MAX98357A	1	Convierte la señal PCM del ESP32 en audio amplificado para la bocina.
Bocina 4 Ω 3W (Ø 40 mm)	1	Reproduce preguntas, lecturas guiadas y correcciones (salida de reproducir_parlante()).
LED difuso 5 mm (verde)	1	Indicador visual: respuesta del niño correcta.
LED difuso 5 mm (rojo)	1	Indicador visual: respuesta del niño incorrecta.
Resistencia 220 Ω	2	Limitan la corriente de los LEDs para proteger su vida útil.
Switch deslizante SPST	1	Interruptor manual de encendido/apagado de la alimentación del circuito.
Cable micro-USB a USB (datos)	1	Suministra 5 V desde la laptop y permite cargar el firmware al ESP32.

Protoboard 830 puntos	1	Montaje y verificación temporal del circuito.
Cables jumper (17 M-M, 4 H-H, 4 M-H)	25	Conectan físicamente los módulos, pines del ESP32, LEDs y switch.
Placa PCB perforada (5×7 cm)	1	Soporte definitivo donde se soldarán los componentes dentro del robot.

Conexiones principales

- **Micro INMP441 → ESP32:** VDD a 3,3 V, GND a GND, SCK al pin de reloj I2S (BCLK), WS al pin de palabra (LRC), SD al pin de datos (DIN) y L/R a GND para seleccionar el canal izquierdo.
- **Amplificador MAX98357A → ESP32:** VIN a alimentación (3,3/5 V), GND a GND y BCLK/LRC/DIN compartiendo el bus I2S con el micrófono.
- **LEDs verde y rojo:** ánodos a pines GPIO a través de una resistencia de 220 Ω; cátodos a GND.
- **Switch SPST:** en serie con la línea de 5 V del micro-USB para el corte total del circuito (un único botón de encendido/apagado según FABLAB).
- El bus I2S del ESP32 soporta la integración simultánea del micrófono y el amplificador sin conflicto de pines (misma señal de reloj, compartida por ambos módulos).

Funcionalidades mínimas y requerimientos

- **Comprensión y captura de voz:** el prototipo capta la voz del usuario mediante el INMP441 (escuchar_micro()). En el MVP se trabaja con lecturas predeterminadas alojadas en el servidor, lo que acota el alcance y permite validar el circuito de voz sin depender aún de la captura por cámara.
- **Salida de audio:** la respuesta generada por la IA llega al amplificador MAX98357A y a la bocina (reproducir_parlante()) para dictar correcciones, agregar tips y generar preguntas de reforzamiento.
- **Interfaz visual y control:** los LEDs verde (correcto) y rojo (incorrecto) dan retroalimentación inmediata; el sistema alterna entre el modo escucha y el modo parlante (operación half-duplex) para evitar el eco/acople acústico, y el switch SPST controla la alimentación.

Componentes del prototipo

- **MVP:** placa ESP32 NodeMCU, micrófono (sensores) y altavoz (actuador).
- **Final:** Raspberry Pi 5 8GB, micrófono y cámara (sensores) y altavoz y pantalla (actuadores), con batería recargable y solar.

Funcionalidades

- **Interacción dinámica — lectura guiada por voz/audio:** el robot capta la voz del usuario a través del micrófono, la envía al servidor, la procesa con la IA y responde por audio con el altavoz.

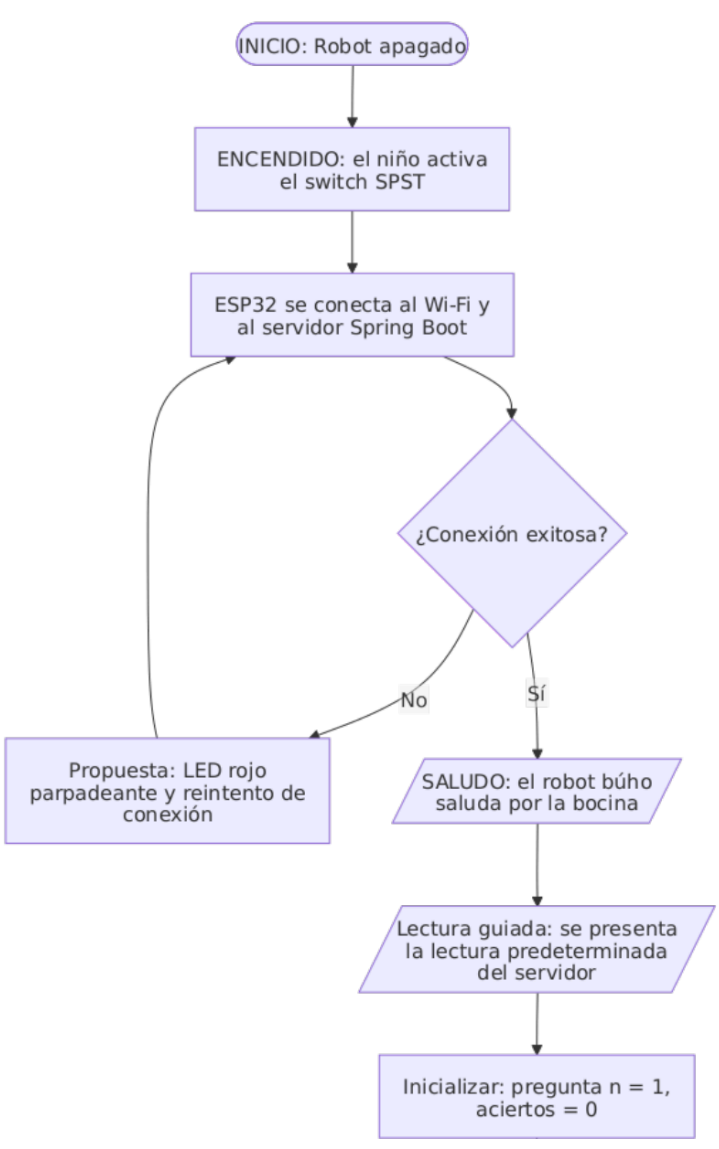
- **Evaluación gamificada de comprensión:** tras la lectura, el robot plantea preguntas, corrige errores, refuerza los aprendizajes y emplea ejemplos, manteniendo una experiencia similar a un juego.
- **Aprendizaje adaptativo (versión final):** módulos que ajustan la dificultad según el desempeño y vocabulario del niño, sin necesidad de conexión a internet.

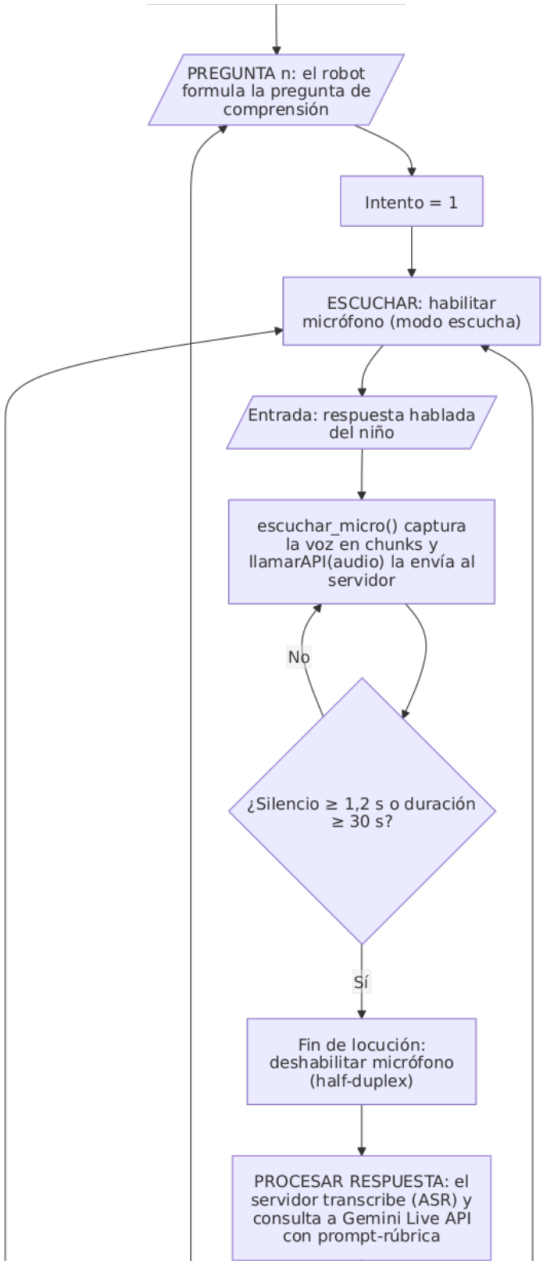
Flujo de datos del MVP

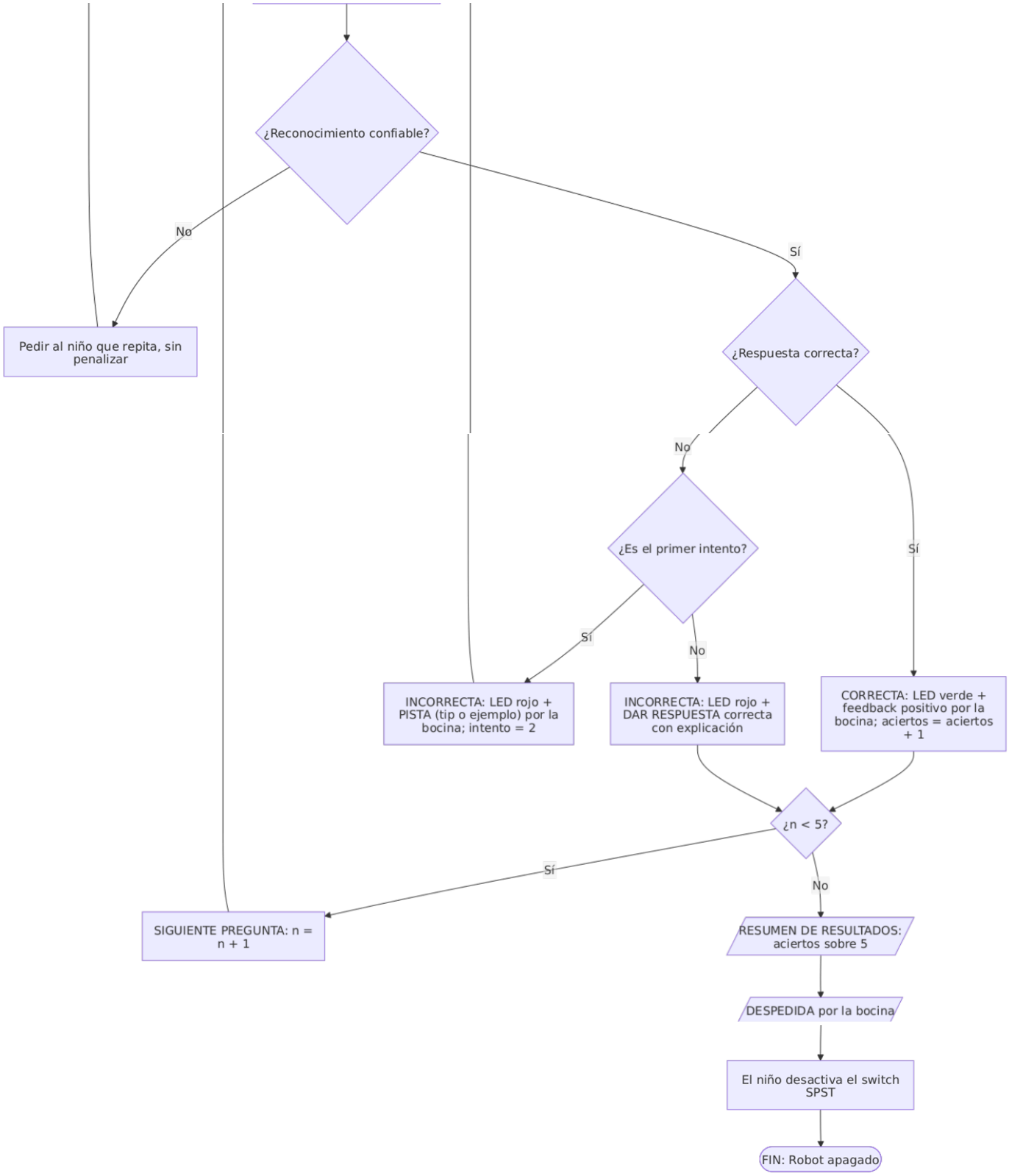
1. El niño inicia la lectura guiada y el ESP32 capta su voz mediante **escuchar_micro()**.
2. La función **llamarAPI(audio)** envía el audio al servidor Spring Boot (computadora que aloja el servidor).
3. El servidor consulta a la API de IA (Gemini Live API) y procesa la respuesta.
4. El servidor convierte la respuesta a audio (TTS) y la regresa al ESP32.
5. El ESP32 la recibe con **escucharAPI()** y la reproduce con **reproducir_parlante()**.
6. El ciclo se repite para las preguntas de comprensión y la retroalimentación gamificada.

De esta manera, el servidor funciona como puente entre el hardware del robot y la inteligencia artificial, asegurando una experiencia por voz fluida y escalable hacia la versión final.

Flujograma del proyecto




PREGUNTA n: el robot formula la pregunta de comprensión
Intento = 1
ESCUCHAR: habilitar micrófono (modo escucha)
Entrada: respuesta hablada del niño
escuchar_micro() captura la voz en chunks y llamarAPI(audio) la envía al servidor
No
¿Silencio ≥ 1,2 s o duración ≥ 30 s?
Sí
Fin de locución: deshabilitar micrófono (half-duplex)
PROCESAR RESPUESTA: el servidor transcribe (ASR) y consulta a Gemini Live API con prompt-rúbrica


¿Reconocimiento confiable?
No
Sí
Pedir al niño que repita, sin penalizar
¿Respuesta correcta?
No
Sí
¿Es el primer intento?
Sí
No
INCORRECTA: LED rojo + PISTA (tip o ejemplo) por la bocina; intento = 2
INCORRECTA: LED rojo + DAR RESPUESTA correcta con explicación
CORRECTA: LED verde + feedback positivo por la bocina; aciertos = aciertos + 1
¿n < 5?
Sí
No
SIGUIENTE PREGUNTA: n = n + 1
RESUMEN DE RESULTADOS: aciertos sobre 5
DESPEDIDA por la bocina
El niño desactiva el switch SPST
FIN: Robot apagado

5. Limitaciones de los componentes

5.1. Hardware (GARAGE)

- **Memoria y almacenamiento:** la SRAM (~520 KB) y la flash (4 MB) del ESP32 impiden almacenar o procesar audio completo. Las grabaciones y las respuestas deben enviarse y reproducirse por fragmentos (streaming PCM16), nunca como archivo completo en el microcontrolador.
- **Códec de audio:** el ESP32 no decodifica MP3 de forma eficiente; el servidor debe entregar PCM16 crudo (WAV/linear16) para reproducirlo directo por el bus I2S, sin librerías pesadas en el MCU.
- **Micrófono INMP441:** captura mono sin cancelación de eco y es sensible al ruido ambiental. Requiere el modo half-duplex (escucha/parlante) y, en la versión final, procesamiento de ruido del lado del servidor.
- **Amplificador y bocina (4 Ω 3W):** banda de audio limitada a voz; necesita alimentación de al menos 5 V y a volumen alto distorsiona. El consumo pico condiciona la fuente y el dimensionado de la batería recargable/solar de la versión final.
- **Alcance de conexión:** el MVP depende de Wi-Fi hacia el servidor local; en zonas sin red no alcanza a Gemini. Se resuelve en la versión final con IA local, según la migración al Raspberry Pi 5 (sección 7).

5.2. Servidor y modelo de IA (CS)

- **Conectividad y dependencia:** Gemini Live API exige internet; fuera de línea no hay respuestas de IA. Este límite se atiende en la versión final con IA local (migración al Raspberry Pi 5, sección 7).
- **Cuotas y costo:** límites de peticiones por minuto y costo por token. Con varios niños simultáneos se necesita cola de peticiones, caché de respuestas y control de presupuesto.
- **Latencia:** ASR + LLM + TTS agregan ~1–3 s por turno; un niño percibe como falla un silencio mayor. Se mitiga con TTS caché de lecturas y streaming parcial.
- **Concurrencia del servidor:** Spring Boot maneja peticiones bloqueantes por defecto; las solicitudes de audio largas pueden agotar el thread pool. Se usarán llamadas reactivas/WebSocket con timeouts.
- **Seguridad de la clave:** la API key va en variable de entorno del servidor; nunca en el firmware del ESP32 ni en repositorios.
- **Privacidad y datos:** el audio y el progreso de menores son datos personales (Ley 29733). Se exige consentimiento de apoderados, anonimización y retención mínima.

6. Identificación de problemas no previstos y mitigación

Se identificaron riesgos técnicos que pueden surgir al implementar el prototipo. Para cada uno se plantea una solución ágil que no limita el alcance del proyecto.

Riesgo	Impacto	Mitigación
La respuesta de audio de la IA supera la memoria del ESP32.	Bloqueo o reinicio; pérdida de la respuesta.	Streaming PCM16 por chunks con I2S+DMA y ring buffer (~16–24 KB de RAM fija); nunca almacenar el audio completo en el MCU; PSRAM como respaldo opcional.

Formato de audio incompatible (MP3 del TTS vs. hardware I2S).	Reproducción fallida o ruido.	El servidor transcodifica a PCM16 con ffmpeg (o reenvía LINEAR16 de Gemini Live); el ESP32 reproduce PCM16 por I2S sin decoder.
Eco/acople cuando el robot habla y escucha a la vez.	Devolución de audio; respuestas erróneas.	Máquina de estados en el firmware que deshabilita el I2S RX (micrófono) durante la reproducción; el servidor opera en request/response y el VAD respalda el corte.
El robot no detecta cuándo termina de hablar el niño (VAD).	Esperas infinitas o cortes de respuesta.	Detección de fin de locución por RMS en el firmware: silencio de 1,2 s o máximo 30 s; VAD de WebRTC en el servidor como segunda instancia.
ASR imperfecto en niños (voz infantil, acentos).	Falsos positivos/negativos en la evaluación.	Gemini evalúa el audio directamente con un prompt-rúbrica (fluidez, articulación, comprensión) + heurística de palabras por minuto; si la confianza es baja, pedir repetir sin penalizar.
Idioma natal (quechua/aymara) mal soportado por ASR/TTS.	Pérdida del público objetivo.	MVP en español; el idioma es un parámetro de configuración del servidor (no código), con hoja de ruta por idioma y evaluación de los ASR/TTS disponibles.

Estos riesgos se revisarán conforme avance la implementación; la mayoría se atendieron desde el diseño (memoria, códec y latencia) para no condicionar las decisiones del proyecto.

Factibilidad de las soluciones en el ESP32

Para responder a la validación técnica de las soluciones propuestas, se detalla con cifras por qué el streaming de audio es implementable en el ESP32 y no depende de recursos que la plataforma no posee.

- **Memoria y streaming:** a 16 kHz, 16 bits y 1 canal, el audio equivale a 32 KB/s. El ESP32 captura con I2S + DMA escribiendo en un ring buffer (doble buffer de ~2×8 KB); cada chunk (~100 ms = 3,2 KB) se envía al servidor apenas se llena, por WebSocket/HTTP chunked, sin acumular audio. La RAM fija de audio es de ~16–24 KB, muy por debajo del heap libre típico de una aplicación con Wi-Fi (200–300 KB). Una respuesta típica de la IA (< 8 s ≈ 256 KB) podría incluso caber en RAM, pero se opta por streaming por robustez ante respuestas largas; PSRAM queda como respaldo con una placa superior.
- **Reproducción sin decoder:** el servidor entrega PCM16 crudo por chunks; el ESP32 los encola (~256–512 ms) y alimenta el MAX98357A por I2S DMA de forma continua. Si la red tarda, se inserta silencio de padding (falta breve de audio, sin distorsión ni reinicios). Es el mismo patrón de un reproductor de radio por internet en ESP32, ya probado en la plataforma.
- **Latencia oculta por diseño:** las lecturas predeterminadas se convierten a PCM16 y se cachean en el servidor al arrancar (0 latencia percibida al dictarlas); el ASR corre en streaming mientras el niño lee, de modo que la transcripción ya está lista cuando termina; sólo la evaluación final depende del LLM (~1–2 s), y el TTS de la siguiente pregunta se pre-genera en paralelo.
- **Speech-to-Text → Text-to-Speech cerrado:** el firmware sólo ve PCM16 de entrada y salida; el ESP32-UART-transcodificación (ffmpeg), ASR (Vosk) y LLM/TTS (Gemini, luego

Ollama/Piper) residen todos en el servidor, donde hay CPU, memoria y la clave de la API protegida.

7. Conclusiones y siguientes pasos

Conclusiones:

- La solución ataca la problemática desde la raíz, trabajando con niños que ya saben leer para que desarrollen la comprensión, interpretación y redacción de textos desde la etapa escolar.
- La arquitectura por capas (ESP32 → servidor → IA) separa responsabilidades: el hardware se encarga de la interacción por voz y el servidor del procesamiento inteligente, lo que garantiza seguridad (protección de la clave de la API) y viabilidad técnica con recursos limitados.
- El MVP valida las funcionalidades esenciales —lectura guiada por voz y evaluación gamificada— antes de invertir en el hardware más potente de la versión final.
- Las condiciones de FABLAB (búho amigable, botón único y energía recargable y solar) orientan el diseño para que el niño use el robot de forma autónoma y atractiva.
- El análisis de limitaciones y riesgos (secciones 5 y 6) permite planear soluciones desde el diseño —streaming PCM16, half-duplex, TTS caché y arquitectura híbrida— en lugar de descubrir los problemas durante la implementación.

Siguientes pasos:

- Validar el MVP con niños del público objetivo y ajustar la calidad del reconocimiento de voz y de la respuesta de la IA.
- Migrar al Raspberry Pi 5 8GB e integrar la cámara para la lectura de textos en formato físico.
- Incorporar la pantalla y los módulos de aprendizaje adaptativo.
- Implementar la batería recargable y solar y la carcasa final tipo búho según el prototipo de FABLAB.
- Lograr la operación 100% offline con base de datos local que registre la experiencia y progreso de cada niño.
- Completar el flujograma del proyecto con inputs/outputs y validar en conjunto la tabla de riesgos de la sección 6.